HARM v4

What is new with respect to HARM v3

Samuele Germiniani
University of Verona

Technical report, October 2026

Abstract

HARM is a hint-based miner of temporal assertions from simulation traces. Version 4 is the result of a plan of 29 milestones carried out between v3 and v4. It makes HARM's output deterministic and its SystemVerilog output valid; extends the proposition language with SystemVerilog syntax and documents its semantics on unknown values; adds an option for the end-of-trace semantics of a simulator; removes semantically redundant assertions, proving equivalence of propositions with Z3 and implication between assertions with Spot together with an exact model of HARM's finite-trace evaluation; and lets the RTL guide mining through cones of influence, computed by a new generator, `harm-coi`, which also harvests the RTL's own predicates. Every new feature is opt-in. This report describes each change, why it was made, the decisions behind it, and how it was validated: every semantic component was checked against an oracle that shares none of its code, and most oracles were themselves checked by planting bugs.

Contents

1 Introduction **3**
- 1.1 HARM v3 in one page . . . 3
- 1.2 Why v4 . . . 3
- 1.3 What is new . . . 3
- 1.4 How the work was done . . . 4

2 Changes to the default output **5**
- 2.1 Deterministic output (D-001, H0) . . . 5
- 2.2 SystemVerilog output (D-002, H1) . . . 5
- 2.3 The meaning of `->` in SVA, and brackets in Spot LTL (D-021, H1d) . . . 6
- 2.4 Vector indices follow SystemVerilog (D-028, H11c) . . . 6
- 2.5 Bug fixes that change results . . . 7

3 The proposition language and its semantics **7**
- 3.1 SystemVerilog syntax (H1) . . . 7
- 3.2 Unknown values: HARM's rule, documented (D-011, H1b) . . . 7
- 3.3 The end of the trace (D-015, D-016, H1c) . . . 8

4 Semantic redundancy reduction **9**
- 4.1 Equivalent propositions (H2, D-003) . . . 9

4.2 Implied assertions (H3, D-004, D-008) . . . 10
4.3 Facts between comparisons (H3b, D-025) . . . 11

5 Cones of influence **12**
5.1 The contract: `coi.json` (H4, D-005, D-013) . . . 12
5.2 Rank mode (H6, D-014) . . . 13
5.3 Filter mode, signal level (H7, D-017, D-018) . . . 13
5.4 Filter mode with depths (H8, D-007, D-020) . . . 14
5.5 The out-of-cone report (H9, D-022) . . . 15

6 `harm-coi`: cones and predicates from the RTL **15**
6.1 The front end (H5a, D-006) . . . 15
6.2 Computing cones (H5, D-019) . . . 15
6.3 RTL predicates (H10, D-023) . . . 16
6.4 Hints from the RTL (H22, D-036) . . . 17
6.5 Relations from the RTL (H24, D-038) . . . 17
6.6 Signed RTL (H25, D-039) . . . 18
6.7 Constants outside the reset branch (H26, D-040) . . . 18
6.8 Elaboration (H23, H28) . . . 18

7 Determinism and portability **19**
7.1 One toolchain (D-010, H0) . . . 19
7.2 Linux (H11b, H11c, H11f) . . . 19
7.3 The test tools in `third_party` (H11d, H11e, H11g) . . . 20
7.4 The same assertions on arm64 and x86_64 (H12, F-M2) . . . 20
7.5 Smaller changes . . . 21

8 Validation **22**
8.1 Testing the oracles . . . 24
8.2 Tests that were wrong . . . 24
8.3 The test suite . . . 25
8.4 Not validated . . . 25

9 Limitations and future work **25**

A Reference **26**
A.1 New command-line options . . . 26
A.2 The options of `harm-coi` . . . 26
A.3 New configuration elements and metric variables . . . 27
A.4 Decisions . . . 27
A.5 Milestones and findings . . . 29

1 Introduction

1.1 HARM v3 in one page

HARM [6] mines temporal assertions from the simulation traces of a design. It does not read the design: its inputs are one or more traces (VCD, sampled at the rising edges of a clock, or CSV, one row per cycle) and a configuration of *hints* written by the verification engineer:

- **propositions** (`<prop exp="cnt == 4'd9"/>`): Boolean expressions over the trace's variables, in C or SystemVerilog syntax;
- **numerics** (`<numeric>`): arithmetic expressions from which HARM generates propositions by clustering their values [5];
- **templates**: temporal formulas with placeholders, e.g. `G({..#1&..} |-> P0)` or `G(P0 -> X P1)`;
- **metrics**: formulas over the features of an assertion (its contingency table `atct`, `atcf`, ..., its complexity, its fault coverage), used to filter and rank.

Propositions are grouped into *domains* (`loc="a"`, `"c"`, `"dt"`, or local numbers), which restrict the placeholders they may fill. A *context* groups the hints that are mined together.

Mining runs on three levels of parallelism. For each template (level 3), HARM enumerates the *permutations* that fill its placeholders with propositions of the right domains (level 2). A template may contain one *decision-tree operator* in its antecedent (`..&&..`, `..##1..`, `..#1&..`): for each permutation, a greedy decision tree then grows the antecedent one proposition at a time, keeping the candidates that best separate the cycles where the consequent holds from those where it fails (level 1). Every candidate assertion is evaluated on the traces with a deterministic automaton built by Spot [3]; an assertion is kept if no instance of it fails inside the trace. The kept assertions are qualified: trivial and duplicated ones are removed, metrics filter and rank them, optional edit rules rewrite them, and they are printed in Spot LTL, PSL or SVA.

1.2 Why v4

HARM v4 was developed for two uses.

- **trivergence**, a project that compares three views of a design: the specification (through an LLM), the RTL, and the simulation traces, with HARM as the trace view. It needed HARM's output to be deterministic, valid SystemVerilog, free of redundant assertions, and able to take structural hints from the RTL. Its adapter worked around several of HARM's limits; v4 removes most of those workarounds.
- **Traditional RTL-guided mining**, as GoldMine [10] does: restricting the search to what the RTL can produce. GoldMine does not support SystemVerilog; v4 can do this for SystemVerilog designs with `harm-coi`.

1.3 What is new

- **Default output** (Section 2): deterministic order; valid SystemVerilog; correct brackets in Spot LTL; SystemVerilog vector indices; bug fixes.
- **Proposition language and semantics** (Section 3): SystemVerilog literals and operators, invariant templates, skipping invalid propositions, HARM's rule for unknown (`x`/`z`) values, and the end-of-trace semantics of a simulator as an option. Operators also bind, evaluate and print as in SystemVerilog, with its widths, signedness, `$signed` and `$unsigned`, and a signal may be named after a HARM keyword (Section 7.5).

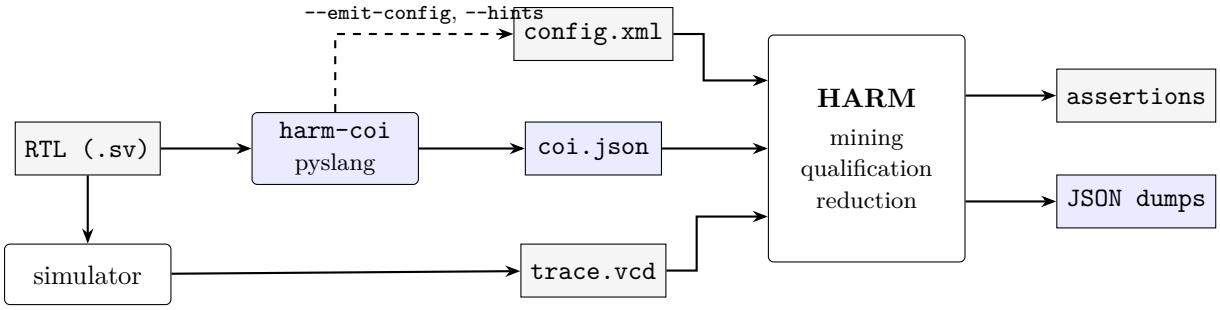


Figure 1: The tool flow of HARM v4. New parts are shaded. The RTL is optional: without it, HARM works as in v3 on traces and hints only.

- **Semantic redundancy reduction** (Section 4): equivalent propositions (Z3), implied assertions (Spot and an exact model of HARM's evaluation), facts between comparisons, and a bound on the pairs checked.
- **Cones of influence** (Section 5): a file format, a rank mode that scores assertions by structural plausibility, a filter mode that prunes the search, with or without depths, and a report of the propositions outside each cone.
- `harm-coi` (Section 6): a generator of cones, RTL predicates and hints from SystemVerilog: signal roles, where each proposition may go, templates per kind of target, and relations that restate the RTL's assignments (signed arithmetic included), which let HARM catch 47 of the 49 bugs of the miner portfolio's D1 corpus.
- **Interfaces for other tools**: JSON dumps of each kept assertion (with the implication step's record), of the dropped ones, of the out-of-cone report and of every proposition's value at every cycle (Appendix A), so that trivergence and the miner portfolio need no VCD reader or evaluator of their own.
- **Determinism and portability** (Section 7): one toolchain, Linux and macOS with identical results, the test tools in `third_party/`, profiling as an option.
- **Validation** (Section 8): the oracles behind each component, and the tests of the oracles.

Section 9 lists the known limitations. The appendix is a reference: the new options and XML elements, every decision, and every milestone with its findings.

Figure 1 shows how the parts fit. HARM stays independent of the design's source: the RTL is read only by `harm-coi`, which talks to HARM through a versioned JSON file.

1.4 How the work was done

The changes were made in 39 milestones, H0 to H30 (H20 is future work) (Table 7 in the appendix), under rules that this report refers to throughout.

1. **Default behaviour is unchanged unless a decision says otherwise.** Every feature is opt-in. Regression baselines of all the examples were frozen in H0; any change to them is recorded as a decision (D-*nnn*, Table 6).
2. **Tests first.** Each milestone's acceptance tests were committed failing before the implementation. A test was never weakened to pass; where an expectation turned out wrong, the change and its reason were recorded.
3. **Independent validation.** Each semantic component (equivalence, implication, cones, printing) is checked by an oracle that does not share its code: brute-force enumeration, a

simulator, Spot, a model written from the IEEE standard, or hand-labelled fixtures. Where possible, bugs were planted to check that the oracle catches them (Section 8).

4. **Two platforms.** macOS on arm64 and Linux on x86_64 must pass the same tests and mine the same assertions.

The development record is in `doc/plan/`: the plan (`doc/plan/PLAN.md`), one plan per milestone, the decisions (`doc/plan/DECISIONS.md`) and the validation log (`doc/plan/VALIDATION.md`). This report summarises them; each claim cites the decision or milestone it comes from.

2 Changes to the default output

A v3 user who runs v4 with the same configuration and no new option sees the changes of this section, and no other. Each is a decision recorded in `doc/plan/DECISIONS.md`, with the regression baselines updated once and the change checked to be exactly the one intended.

2.1 Deterministic output (D-001, H0)

In v3, the same input could give a different output from run to run, and with a different number of threads. Worse, the `--max-ass` cut and the fault-coverage subset of `--find-min-subset` could change *which* assertions were reported, not only their order. Eight of seventeen determinism tests written in H0 failed on v3. Five sources were found:

1. the miner appended each permutation's assertions in the order in which threads finished;
2. the redundancy filter iterated a hash set keyed by pointers, whose order depends on addresses even with one thread;
3. the ranking sorted on the score only, so equal scores came out in an arbitrary order;
4. the fault-coverage code iterated maps keyed by assertion identifiers, which come from a global counter incremented as threads create assertions;
5. the faulty traces of `--fd` were read in directory order and then shuffled with a random seed.

v4 collects the results per (template, permutation) in key order, deduplicates in input order, ranks by final score and then by the assertion's text, sorts the set-cover candidates by text, and shuffles the faulty traces with a fixed seed. The output is identical across runs and thread counts on every regression case, including the `--max-ass` cuts (19 determinism tests in H0, 29 today).

2.2 SystemVerilog output (D-002, H1)

v3's `--sva` output was not SystemVerilog that tools accept: it printed `true`, the package-scope separator `::` for hierarchical names, and `nexttime`, which Verilator [9] and EBMC reject. v4 prints:

v3	v4
`always (Reject_Application |-> nexttime Assess_Loan_Risk)`	`always (Reject_Application |=> Assess_Loan_Risk)`
`always (Store_feedback |-> eventually Assess_Eligibility)`	`always (Store_feedback |-> s_eventually Assess_Eligibility)`
`always (!intf::a ##1 true |-> DUT::state == 2'b0)`	`always (!intf.a ##1 1'b1 |-> DUT.state == 2'b0)`

These are real lines from the `process` and `fsmSVT` examples, printed by v3 and v4. The rules: `true`/`false` become `1'b1`/`1'b0`; `p |-> nexttime q` becomes `p |=> q` and `nexttime[n]` becomes `##n` when `q` is Boolean; unbounded `eventually`, which is not SystemVerilog, becomes `s_eventually`; `a::b` becomes `a.b`. `H1c` also found that v3 printed SVA with HARM's own operator precedence, so `(b W c) && X X F b` came out as `b until c and nexttime nexttime s_eventually b`, which SystemVerilog parses as `b until (c and ...)`. v4 adds the brackets that SystemVerilog's precedence (IEEE 1800-2017, Table 16-3 [7]) needs.

User-written `<edit>` rules are matched against the printed text. To keep existing rules working, HARM matches them against v3's printing and prints the result in v4's.

```
$ harm --csv-dir examples/process/traces --conf examples/process/processConfig.xml --sva \
    --dont-print-ass --dump-to $OUT/process.txt > /dev/null
$ grep -c . $OUT/process.txt
138
$ grep -E 'Reject_Application \|=> Assess_Loan_Risk' $OUT/process.txt
always (Reject_Application |=> Assess_Loan_Risk)
```

2.3 The meaning of -> in SVA, and brackets in Spot LTL (D-021, H1d)

Two printing bugs, found while checking cone depths in `H8` (findings F7 and F8 there), changed the *meaning* of printed assertions.

F7: `->` with a multi-cycle antecedent. HARM's `->` starts the consequent together with the antecedent, as in Spot and PSL; SVA's `|->` starts it at the antecedent's *end*. They agree only when the antecedent lasts one cycle. v3 printed `G({a ##1 b} -> X c)` as `a ##1 b |=> c`, which puts `c` one cycle late. v4 re-anchors the consequent at the end of the antecedent: for an antecedent of fixed length n cycles and a Boolean consequent after k cycles of `X` (plus one for `=>`), with $j = k - (n-1)$, it prints `s |-> c` ($j = 0$), `s |=> c` ($j = 1$), `s |-> ##j c` ($j > 1$) or `s |-> $past(c, -j)` ($j < 0$). When that is not possible (a variable-length antecedent, or a non-Boolean consequent), it prints IEEE 1800's `implies`, which starts both sides together. Spot's `ltlfilt` confirms that the new texts are equivalent to the formulas HARM mined and the old ones are not.

F8: brackets in Spot LTL. Spot binds `X`, `F`, `!`, `U`/`W` and `R` tighter than `&&` and `||`. v3 printed `X(en && wrap)` as `Xen && wrap`, which Spot reads as `(X en) && wrap`. v4 brackets a proposition whose top operator is a Boolean connective under those operators. In the `sub_platform` example, 46 of the 91 assertions change this way, and nothing else changes.

2.4 Vector indices follow SystemVerilog (D-028, H11c)

v3 stopped on a VCD vector declared with an ascending range, such as `input [1:10] state` (finding F-L4, on the AssertLLM2 design `ethernet_smii_txrx`), and always counted bit positions from the right from 0. v4 keeps each vector's declared range `[l:r]`: `x[i]` is bit $|i - r|$ from the right, an index outside the range is an error, and so is a part-select against the declared direction. Vectors declared `[n:0]` and CSV variables keep their meaning, so no baseline changed. The motivation was `harm-coi`, which writes the RTL's indices: before D-028, its predicates on vectors not declared `[n:0]` were misread. A Verilator testbench prints 14 selects on four vectors every cycle, and HARM's values on the same trace equal Verilator's on all 41×14.

2.5 Bug fixes that change results

- **F10 (H1):** copying a bit selection swapped its bounds, so a mined `r[7:4]` was printed and evaluated as `r[4:7]`. Every mined assertion is a copy of its template, so this affected every assertion with a part-select.
- **F9 (H1):** variable names were substituted into propositions without identifier boundaries, so a variable `a` or `x` corrupted the literals `0xa` or `'b1x0`.
- **H7:** a template with more placeholders of one kind than propositions in their domain made HARM hang: a binomial coefficient with $k > n$ recursed in exponential time.
- **F-M2 (H12):** on macOS arm64, HARM could mine different assertions than on Linux x86_64 (Section 7.4).

3 The proposition language and its semantics

3.1 SystemVerilog syntax (H1)

v3 propositions accepted C operators, binary literals (`4'b1010`, `0b...`, `0x...`), bit selection, `inside` and the SVA functions. Several forms that trivergence's specification-derived propositions use were lexer errors, and one bad proposition aborted the whole run. v4 adds:

Form	Example	Notes
based literals	`8'd9`, `4'hA`, `6'o7x`, `8'sd5`	`x`, `z`, `?` digits; `_` separators; unsized `'h`/`'d`/`'o` are 32 bits
fill literals	`q4 == '1`	`'0 '1 'x 'z` take the width of the other operand
concatenation	`{go, s} == 3'b110`	items are logic, int or bool variables
replication	`{4{1'b0}}`	
conditional	`(go ? cnt : 4'h0) > 4'd5`	lowest precedence, as in SystemVerilog
case equality	`q4 === 4'b1x0z`	compares `x` and `z` exactly
hierarchical names	`u_core.state`	the same as `u_core::state`

Each new node needed an evaluator, a printer, a copier, and an encoding for Z3 (Section 4.1). Two more additions:

- **Invariant templates:** `G(P0)`, without an implication, is read as `G(true -> P0)` and printed back as `G(P0)`. v3 rejected it.
- **`--skip-invalid-props`:** a proposition that does not parse is skipped with a warning instead of stopping HARM. This needed a scoped "throw instead of exit" mode around proposition parsing, because `messageError` exits.

The semantics of the new operators was validated against Icarus Verilog [11] on 1,000 random expressions × 32 random 4-valued rows (Section 8).

3.2 Unknown values: HARM's rule, documented (D-011, H1b)

HARM's `Logic` type holds 4-valued bit vectors (`0`, `1`, `x`, `z`) of up to 511 bits, and computes arithmetic, bitwise operators and concatenation with `x` and `z` as SystemVerilog does. The two differ where a 4-valued value becomes a truth value. The H1 oracle measured HARM's rule precisely, and H1b documented it:

- a comparison (`== != < <= > >=`) is false if any bit of an operand is `x` or `z`;
- a value used as a condition is true only if it has a known `1` bit;
- `!`, `&&` and `||` then work on true and false;
- `===` and `!==` compare `x` and `z` exactly.

In SystemVerilog [7, §11.4.4, §11.4.5, §11.4.7, §16.6], a comparison or condition can be `x`, the Boolean operators propagate it, and an assertion treats a final `x` as false. On cycles where a signal has unknown bits the two can disagree in both directions. With `a = 4'b10x1`, `b = 4'b1001`, `c = 4'b0001`, `p = 1'bx`:

Proposition	HARM	SystemVerilog	Consequence
`!(a == b)`, `!p`, `p || !p`	true	false (`x`)	HARM may mine what a simulator sees fail
`a != c` (a known bit differs)	false	true	HARM may miss what holds in simulation
`a == b`, `a === b`, `a !== b`	the same		

The regression `h1b_x_semantics` pins HARM's value of 23 such propositions; their SystemVerilog values were checked against the standard and with Icarus Verilog, and differ on 9 of the 23.

Why the rule was kept (D-011, D-024 withdrawn). An option with SystemVerilog's semantics was planned (D-024) and then withdrawn by decision. In HARM a proposition is true or false, and a proposition and its negation are complementary. The decision trees (negated candidates, negated consequents) and the reductions of Section 4 rely on that. Under SystemVerilog's semantics, both `a == b` and `!(a == b)` are false on a cycle where `a` has an `x` bit. The difference matters only on traces with unknown values: before reset in a 4-state simulator, uninitialised memories, tri-state nets. The README tells users to mine after reset and to write `===`/`!==` where unknown values must be told apart.

3.3 The end of the trace (D-015, D-016, H1c)

A finite trace ends while some assertion instances are still pending: `G(a -> F b)` when the last `a` has not yet been followed by `b`. HARM has always counted a pending instance as holding, the *weak view* of truncated paths [4]. A SystemVerilog simulator follows the *neutral view* at the end of a simulation: a pending weak obligation holds, a pending strong one fails. Since v4 prints unbounded `F` as `s_eventually`, which is strong, a liveness assertion HARM accepts could fail in a simulator on the same trace.

`--trace-end` chooses:

- `harm` (default): every operator is weak at the end, as in v3;
- `sva`: weak obligations (`nexttime`, `until`, `always`, sequences) hold, strong ones fail: `s_eventually p` (`F p`), `not nexttime p` (`!X p`, which is `s_nexttime !p`) and `not (p until q)`.

Only verdicts that are pending at the end can change, and only from holding to failing. The consequent is evaluated with Spot's LTLf translation [1] (`from_ltlf`), which also says in which automaton states the trace may end. Spot 2.9.7 reads `X` as weak there, which a first implementation got wrong and the hand-derived tests caught. The default stays `harm` because the change would affect other users: 106 of the 138 assertions of the `process` example contain a strong operator, and with `sva` 62 of them are not mined.

```
$ harm --csv-dir examples/process/traces --conf examples/process/processConfig.xml --sva \
    --trace-end sva --dont-print-ass --dump-to $OUT/sva.txt > /dev/null
```

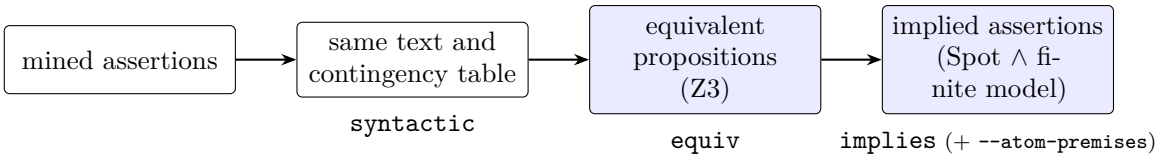


Figure 2: The reduction levels. Each level includes the previous ones; the shaded ones are new in v4.

```
$ grep -c . $OUT/sva.txt
76
```

The oracle for this option is a model of IEEE 1800's finite-trace semantics written independently of HARM, which parses HARM's printed SVA with the standard's precedence: 297 generated assertions on 4,680 traces in both modes, with no disagreement.

4 Semantic redundancy reduction

Mining with decision trees produces many assertions that say the same thing, or less: `G({a && b} -> c)` next to `G(a -> c)`, or `cnt == 4'd9` next to `4'd9 == cnt`. v3 removed only assertions with the same text and the same contingency table. v4 adds three levels, chosen with `--reduce` (Fig. 2):

- `syntactic` (default): v3's filter;
- `equiv`: assertions that differ only in equivalent propositions are merged (Z3);
- `implies`: in addition, an assertion implied by another kept assertion is dropped (Spot and a model of HARM's evaluator); `--atom-premises` adds facts between comparisons.

The logic was ported from USM-T, a fork of HARM's core, and rebuilt where HARM's semantics required it. Reduction runs after the `<filter>` metrics and the edit rules, and before ranking, so that the kept assertions are ranked as usual.

4.1 Equivalent propositions (H2, D-003)

Encoding HARM's evaluator in Z3. Two propositions are merged only if Z3 [2] proves them equivalent under HARM's semantics. The first proposal, "never merge propositions with `x`/`z` constants or `===`, use two-valued logic for the rest", is unsound: variables can hold unknown values at run time, and under HARM's rule (Section 3.2) `a == c` and `!(a != c)` then differ. So v4 encodes HARM's evaluator exactly (D-003): each logic signal is a value bit-vector plus an `x` mask and a `z` mask, and each operator follows HARM's implementation of `Logic`. Implementing it uncovered details that the encoding had to reproduce: hidden value bits above a value's width (the carry of `+`, the ones of `~`), `~` turning `z` into `x`, arithmetic with an unknown operand giving a 1-bit `x`. Constructs that are not encoded exactly (division and shifts, `$past` and the other SVA functions, strings, integer types narrower than 64 bits) become *opaque atoms*, identified by their text: sound, but incomplete. A timeout or an *unknown* answer means "not equivalent".

Canonicalisation. Propositions with the same text are grouped without a solver call; the others are compared with Z3 only within buckets of propositions over the same variables. Each class gets a token, and the redundancy filter's key uses the tokens instead of the text, still together with the contingency table. Of a merged group, the assertion with the smallest text is

kept. `cnt == 4'd6` and `4'd6 == cnt` are merged; `s !== 2'b00` and `s != 2'b00` are not, although they agree on this x-free trace, because they differ when `s` has unknown bits:

```
$ harm --csv tests/input/h1/newops.csv --conf tests/input/h2/reduce.xml --reduce equiv \
    --dont-print-ass --dump-to $OUT/equiv.txt > /dev/null
$ cat $OUT/equiv.txt
G({go, s} == 3'b110 -> s != 2'b0)
G({go, s} == 3'b110 -> s !== 2'b0)
G(4'b110 == cnt -> s != 2'b0)
G(4'b110 == cnt -> s !== 2'b0)
```

Without `--reduce equiv`, the same configuration gives six assertions: the two above with `cnt == 4'b110` are also kept.

Z3 is an optional dependency (D-009): it is built from source in `third_party/` with HARM's compiler, and the CMake option `HARM_WITH_Z3` (default on) can turn it off, in which case `equiv` and `implies` are not available.

4.2 Implied assertions (H3, D-004, D-008)

From assertions to Spot formulas. Each assertion is abstracted to an LTL formula over atoms. The abstraction stops at the maximal non-Boolean subexpressions (comparisons, conversions of logic values to Boolean, function calls): `&&`, `||`, `!` and `^` between them are kept and given to Spot. Otherwise Spot could not see that `G(a -> c)` implies `G({a && b} -> c)`, the most common redundancy in decision-tree output. Atoms are named by their equivalence class from Section 4.1, so `implies` includes `equiv`.

Two semantics, both required (D-004 amended, D-015). The plan was to claim $A \Rightarrow B$ when Spot proves language containment over infinite words and both formulas are syntactic safety: for safety formulas, every finite trace that violates B then violates A, so dropping B loses nothing. The bounded oracle of H3 refuted this on a 3-cycle trace:

$$A = \texttt{G(\{b ##2 !a\} |-> X (b \&\& !b))}, \qquad B = \texttt{G(\{b ##2 !a\} |-> (b \&\& !b))}.$$

Both are false over infinite words, and B fails on the trace. A does not: HARM evaluates the consequent with an automaton over the Boolean *leaves*, which cannot see that `b && !b` is false, so the `X` instance is still pending at the end of the trace, and a pending instance holds. So v4 claims $A \Rightarrow B$ only if it holds in both semantics:

1. over infinite words, with Spot's containment check: this is what SVA, simulators and formal tools need;
2. on every finite trace as HARM evaluates it: an exact model of HARM's evaluator in which every antecedent is a fixed-length sequence of Boolean steps, every consequent is the same deterministic, complete Spot automaton over leaves that HARM builds, an instance starts on every cycle, and an instance still pending at the end holds (or, with `--trace-end sva`, fails if its pending obligation is strong). A breadth-first search over the pending instances of A and B explores every finite trace and refutes $A \Rightarrow B$ if B can fail while A has not.

Only assertions of the form `G(antecedent -> consequent)` with a fixed-length antecedent are reduced; the others (`F`, `[*]`, `##[m:n]`, conjunctions of different lengths) are always kept. Every claim is sound for both trivergence's simulator checks and its formal checks.

Cost. Only pairs of assertions that share a consequent atom, or whose consequent atoms a proved fact links, are compared (H29; before, any shared atom, and the reset that most

antecedents read linked nearly every pair), and every formula is translated to an automaton once. A pair left out is an implication not found, never a wrong drop: $A \Rightarrow B$ without such a link holds only if B is vacuous under A or its consequent follows from its own antecedent. On the miner portfolio's corpus (40 runs) the kept sets are unchanged; on AssertLLM2's `i2c_slave` (6,481 assertions) the pairs fall from 5.7 M to 1.2 M, the time from 479 to 71 s, and 87 more assertions are kept. Above `--implies-max-pairs` pairs (default 2,000,000) the step is skipped for that context, HARM says so, and `--dump-assertion-info` records it. The `camellia` example found a real performance bug: its assertions span 25 cycles (`X[23]`), Spot's automaton for `G(a -> X^k b)` has about 2^k states, and the first finite model tracked all pending instances. Three limits make the reduction terminate, each of which only removes claims: assertions spanning more than 10 cycles are never reduced; the finite search follows one chosen instance of the dropped assertion; and it gives up after 20,000 states ("not proved"). The cost still grows with the square of the number of assertions: on a fixture with 1,839 assertions, `implies` and `--atom-premises` together exceed 30 minutes.

Which side is kept (D-008). `--keep stronger` (default) keeps the implying assertion and drops the implied one; `weaker` does the opposite; `ranked` keeps the better ranked. Equivalent assertions keep the smallest text. `--dump-implications` writes every dropped assertion, the kept assertions that imply it, and the relation:

```
$ harm --csv tests/input/h3/reduce.csv --conf tests/input/h3/reduce.xml --reduce implies \
    --dump-implications $OUT/implications.json > /dev/null
$ cat $OUT/implications.json
{"context": "default", "dropped": "G({a && b} -> c)", "kept": ["G(a -> c)"], "relation": "
    implied"},
{"context": "default", "dropped": "G({a && d} -> c)", "kept": ["G(a -> c)", "G(d -> c)"], "
    relation": "implied"},
{"context": "default", "dropped": "G({b && d} -> c)", "kept": ["G(d -> c)"], "relation": "
    implied"}
```

On this trace `c` is `a || d`: of the five mined assertions, the three conjunctions are implied by `G(a -> c)` or `G(d -> c)`. On a synthetic design with planted relations, 116 mined assertions reduce to exactly the 4 planted ones.

4.3 Facts between comparisons (H3b, D-025)

H3's atoms are opaque to Spot, so `G(cnt > 4'd8 -> b)` does not drop `G(cnt > 4'd9 -> b)`: Spot does not know that `cnt > 9` implies `cnt > 8`. `--atom-premises` asks Z3, for every pair of atoms over a common variable, whether $p \to q$, $q \to p$ and $p \to \neg q$ are valid, with the encoding of Section 4.1 (unknown values included). Only a proof gives a fact. The facts become premises of both checks: Spot checks $\mathbf{G}(\mathit{facts}) \wedge A \subseteq B$, and the finite model explores only atom valuations that satisfy them. Premises can only add implications. The queries are capped by `--atom-premises-max` (default 2,000); beyond the cap HARM says so and the result stays sound, only weaker.

```
$ harm --csv tests/input/h3b/reduce.csv --conf tests/input/h3b/reduce.xml --reduce implies \
    --atom-premises --dont-print-ass --dump-to $OUT/kept.txt \
    --dump-implications $OUT/implications.json > /dev/null
$ cat $OUT/kept.txt $OUT/implications.json
G(cnt > 4'b1000 -> a)
"dropped": "G(cnt > 4'b1001 -> a)", "kept": ["G(cnt > 4'b1000 -> a)"], "relation": "implied",
     "premises": ["cnt > 4'b1001 -> cnt > 4'b1000"]
"dropped": "G(cnt == 4'b1001 -> a)", "kept": ["G(cnt > 4'b1000 -> a)"], "relation": "implied",
     "premises": ["cnt == 4'b1001 -> cnt > 4'b1000"]
```

Without `--atom-premises`, all three assertions are kept. The option was approved after a measurement: on outputs with numeric ranges or FSM states, at least 16–22% of the assertions that survive `implies` are redundant given such facts; on Boolean-only outputs, none. On the `sub_platform` example the output goes from 89 to 69 assertions, and the reduction from 0.9 s to 5.3 s.

5 Cones of influence

HARM mines from traces alone, so it finds correlations that the design cannot produce: an input predicting another input, or an output predicting a register it does not drive. The RTL knows which signals can influence which, and through how many registers. v4 lets HARM use that knowledge without reading the RTL itself: a separate generator (Section 6) writes a *cone-of-influence* file, and HARM reads it in one of two modes.

- **Rank mode** only scores: new metric variables tell how well an assertion agrees with the cones, and can sort or filter. This is for trivergence, where the RTL is one view among three and may be wrong.
- **Filter mode** prunes the search before mining, as GoldMine [10] does. It assumes the RTL is correct: behaviour that an RTL bug removed or added cannot be mined, and HARM says so once.

5.1 The contract: `coi.json` (H4, D-005, D-013)

The file format is versioned (`doc/schemas/coi.v1.json`, JSON Schema 2020-12):

- `meta`: design, top module, the VCD scope and recursion HARM is run with, the clock, `max_depth`, and the generator's name and version;
- `targets`: for every signal HARM sees in the trace, its `sources`, each a list `{sig, depths, saturated}`;
- `unknown`: signals whose cone the generator could not determine;
- `predicates`: the RTL's predicates (Section 6.3).

Depth is the number of register crossings on a path from source to target (D-005): 0 means the source's value now can affect the target now; d means its value d cycles ago. Depths are counted as HARM samples a trace, on the values just before each rising edge, which is what an SVA concurrent assertion sees in the Preponed region. With that convention `q <= a` gives `G(a -> X q)`. A first version of the simulation oracle sampled just after the edge and saw every input-to-register depth one cycle short; HARM's own sampling settled it. All depths up to `max_depth` are listed; `saturated` marks a source that also reaches the target through longer paths. **Targets** are the signals visible in the trace; **sources** are visible signals, with paths through invisible nets followed through; the clock is never a source, and reset is an ordinary one (D-013).

A context names its cone file with `<coi file="design_coi.json" mode="rank"/>`; the path is relative to the configuration. The examples of this section use the `multipath` fixture (`tests/input/coi/multipath/`):

```
always_ff @(posedge clk) begin r1 <= a; r2 <= r1; rb <= b; end
assign y = a ^ r2;      // y <- a at depths 0 and 2, r2 at depth 0
assign z = rb;          // z <- b at depth 1, rb at depth 0
```

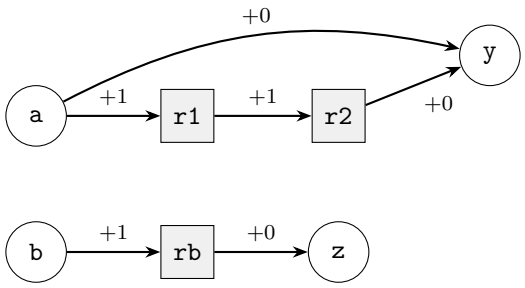


Figure 3: The `multipath` fixture. Edges carry their register delays; the cone of `y` contains `a` at depths 0 and 2, `r1` at depth 1 and `r2` at depth 0. `b` cannot influence `y`.

5.2 Rank mode (H6, D-014)

For a mined assertion `G(antecedent -> consequent)`, let the *leaves* be its atomic propositions, and $\mathit{off}(\ell)$ the cycle at which leaf ℓ is evaluated, relative to the start of the antecedent; *off* is unknown under `until`, `F`, `R`, repetitions and ranges. Let A be the antecedent leaves with variables, C the consequent leaves, and $\mathrm{cone}(c)$ the sources of signal c with their depths. Then

$$\texttt{coiFrac} = \frac{|\{\ell \in A : \forall v \in \mathit{vars}(\ell).\ \exists q \in C,\ c \in \mathit{vars}(q).\ v \in \mathrm{cone}(c)\}|}{|A|},$$

$$\texttt{coiDepthFit} = \frac{|\{\ell \in A : \forall v \in \mathit{vars}(\ell).\ \exists q \in C,\ c \in \mathit{vars}(q).\ \mathit{off}(q) - \mathit{off}(\ell) \in \mathrm{depths}(v, c)\}|}{|A|},$$

where a saturated source also fits at any distance above `max_depth`, and an unknown offset falls back to cone membership. With no antecedent leaves (an invariant), both are 1. `coiUnknown` counts the antecedent leaves with a variable the file does not know; they count as in the cone and fitting: never penalised, but counted.

Offsets depend on the implication: with `|->` the consequent starts at the end of the antecedent, with `|=>` a cycle later, with `->` together with the antecedent. Getting `->` right needed a fix (F6 of H8): the first version anchored it at the end, as for `|->`.

These variables can be used in any `<sort>` or `<filter>`, e.g. `<sort name="fit" exp="coiDepthFit"/>`. On `multipath` with templates `G(P0 -> P1)`, `G(P0 -> X P1)` and `G(P0 -> X X P1)`, the seven assertions that hold score as derived by hand from the RTL; `G(z -> rb)` holds on the trace but `z` cannot influence `rb`:

```
$ harm --vcd tests/input/coi/multipath/trace.vcd --clk clk --vcd-ss tb::dut \
    --conf tests/input/h6/multipath_rank.xml --dont-print-ass \
    --dump-assertion-info $OUT/rank.json > /dev/null
$ python3 -c "import json; [print(a['text'], a['metrics']['coiFrac'], a['metrics']['
    coiDepthFit']) for a in json.load(open('$OUT/rank.json'))['assertions']]"
G(a -> XXr2) 1.0 1.0
G(b -> Xz) 1.0 1.0
G(rb -> z) 1.0 1.0
G(z -> rb) 0.0 0.0
```

Propositions and numerics can also carry a free-text `origin` (`<prop exp="..." origin="spec"/>`), reported by `--dump-assertion-info` and `--dump-coi-report`. trivergence marks the propositions that come from the specification this way.

5.3 Filter mode, signal level (H7, D-017, D-018)

`<coi ... mode="filter"/>` never tries, for a consequent, an antecedent proposition outside its cone. The rule is `coiFrac`'s, so that the two modes share one definition: a proposition is in the cone of a

consequent if *every* one of its variables is a source of *some* variable of the consequent. Unknown signals count as in the cone; propositions without variables and placeholders shared by antecedent and consequent are kept. Permutations of plain templates that use an out-of-cone proposition are removed before mining, and decision-tree candidates are restricted to the consequent's cone. HARM reports the search space before and after, and `--dump-assertion-info` records it.

Not "rank mode minus out-of-cone assertions" (D-018). For plain templates the two are the same. With decision trees, pruning changes what the greedy tree explores, so filter mode can find in-cone assertions that rank mode misses. Its guarantee is the output of rank mode on a configuration from which, for each consequent, the out-of-cone propositions were deleted by hand; that is how it was validated (Section 8).

5.4 Filter mode with depths (H8, D-007, D-020)

`depth="bounded"` or `"exact"` also uses the depths. An antecedent proposition at distance d cycles before a consequent leaf is kept, for each of its known variables v, if some consequent variable c gives:

`depth`	condition
`any` (default)	$v \in \text{cone}(c)$ (the signal-level filter)
`bounded`	$0 \le d \le \max \text{depths}(v, c)$, or v saturated and $d \ge 0$
`exact`	$d \in \text{depths}(v, c)$, or v saturated and $d >$ `max_depth`

`exact` is `coiDepthFit`'s rule; the metric and the filter share one function, so filter output with `exact` has `coiDepthFit` $= 1$. In Fig. 3, `y` gets `a` at depths 0 and 2: `G(a -> X y)` ($d = 1$) is kept by `bounded` and not by `exact`; `G(a -> X X X y)` ($d = 3$) by neither.

Decision trees (D-007). A decision-tree operator adds items at indices; each index is at a fixed distance from each consequent leaf, independent of how many items the tree adds later:

Implication	antecedent layout	distance of index i to leaf q
`|->`	`dM ##N ... ##N d1 ##N d0` (index 0 is the last cycle)	$i \cdot N + \mathit{off}(q)$
`|=>`	the same	$i \cdot N + 1 + \mathit{off}(q)$
`->`	`d0 ##N d1 ... ##N dM` from the start	$\mathit{off}(q) - i \cdot N$

HARM computes this table rather than hard-coding it: it places a marker at each index and runs the same offset function as `coiDepthFit`. A unit test checks 12 template shapes against the table, and mined `counter` assertions were checked by hand. A (candidate, index) pair is tried only if the candidate fits at that distance.

On `multipath`, `exact` keeps 10 of the 147 permutations, and mines the six structural assertions of the rank example, without `G(z -> rb)`:

```
$ sed -e 's#mode="rank"#mode="filter" depth="exact"#' -e 's#\.\./coi/#../tests/input/coi/#' \
    tests/input/h6/multipath_rank.xml > $OUT/filter.xml
$ harm --vcd tests/input/coi/multipath/trace.vcd --clk clk --vcd-ss tb::dut \
    --conf $OUT/filter.xml --dont-print-ass --dump-to $OUT/filter.txt
COI filter (context 'default'): permutations 147 -> 10
$ cat $OUT/filter.txt
G(a -> XXr2)
G(rb -> z)
```

5.5 The out-of-cone report (H9, D-022)

`--dump-coi-report` lists, for each consequent proposition, the antecedent propositions and numerics that cannot influence it, with their `origin` and the signals outside the cone; propositions HARM cannot decide (unknown signals) are listed apart, never as "out". It is computed from the configuration and the cone file, and does not change mining. For trivergence, a proposition with `origin="spec"` outside a cone is a disagreement between the specification ("A influences B") and the RTL ("it cannot").

```
$ harm --vcd tests/input/coi/multipath/trace.vcd --clk clk --vcd-ss tb::dut \
    --conf tests/input/h6/multipath_rank.xml --dont-print-ass \
    --dump-coi-report $OUT/report.json > /dev/null
$ python3 -c "import json; [print(c['text'], '<-', [o['text'] for o in c['outOfCone']]) for c
     in json.load(open('$OUT/report.json'))['contexts'][0]['consequents']]"
y <- ['b', 'rb', 'z']
z <- ['a', 'r1', 'r2', 'y']
```

6 `harm-coi`: cones and predicates from the RTL

`harm-coi` (`tools/harm-coi/`) is a Python tool that reads SystemVerilog RTL and writes the `coi.json` of Section 5. It is the only part of v4 that reads the design; HARM itself stays source-agnostic.

6.1 The front end (H5a, D-006)

Two front ends were compared on the six hand-written fixtures of H4 in a spike: the elaborated syntax tree of pyslang [8], with our own dataflow and control-dependency walk, and the JSON netlist of yosys [12] with its slang front end. pyslang reproduced all six fixtures exactly. yosys needed a version newer than the one packaged on macOS, lost struct fields and parameters, and lowered the conditions that predicate harvesting needs. The decision was **pyslang 12.0.0**, pinned (Python $\geq$ 3.11). yosys stays an optional, signal-level cross-check in the tests.

6.2 Computing cones (H5, D-019)

The front end elaborates the design and extracts, for every assignment, the direct dependencies of its target, with their register delays:

- **data dependencies** from the right-hand side, and **control dependencies** from the conditions of `if`, `case` (selector and items) and `?:`;
- a register assigned in a clocked process has delay 1; combinational logic, continuous assignments and port connections have delay 0;
- a register not assigned on every path holds its value, so it is a source of itself at delay 1;
- **asynchronous controls** (`always_ff @(posedge clk or posedge rst)` where the body reads `rst`) and registers on the falling edge of the sampling clock have depths 0 and 1;
- **blocking temporaries**: a variable local to a process is replaced by its sources; a visible signal assigned once in a process and read later in it is also a source at delay 0. This last rule came from a test that was wrong (H5, F2): the yosys cross-check and the simulation oracle both showed that forcing such a temporary changes the output, so filter mode must not prune `G(t -> y)`.

`cone.py` maps the design's signals to the names HARM sees: relative to the VCD scope, with `::` between sub-scopes. With `--vcd`, the trace decides which signals are visible: a packed struct dumped as one vector is one signal, the union of its fields. The closure up to `--max-depth` sums the delays along paths and marks saturation.

Too large, never too small. A cone that is too small would make filter mode prune real behaviour; one that is too large only prunes less. So where the RTL leaves a doubt, the signal goes to `unknown`, which HARM treats as in every cone: latches (including an `always_comb` output not assigned on every path, since `unique` and `priority` are not trusted to make a `case` complete), registers on another clock or on both edges, `inout` ports, tasks, a driver that slang reports but the walk does not model, and every target whose cone reaches an unknown signal within `--max-depth`.

6.3 RTL predicates (H10, D-023)

`--predicates` also harvests the RTL's own conditions as HARM propositions, written in HARM's syntax and names, with `origin="rtl"`:

Kind	from	predicate
condition	`if (c)`, `c ? x : y`	`c`, and each atom of a compound `c`
case label	`case (s) L:`	`s == L`
enum (FSM) value	a variable of an enum type	`v == C` for every value
comparison	with a constant side, anywhere	the comparison
reset value	the first `if` of a clocked process whose branch assigns only constants	`reg == value`

Constants are written as sized decimal literals of the signal's width (`state == 2'd1`); a 1-bit comparison as the signal or its negation. Predicates on invisible signals, on local variables, or on signals wider than HARM's 511-bit limit (finding F-L5, on the AssertLLM2 design `sha3`) are dropped and counted. `--emit-config` writes a ready-to-run HARM configuration: one context with the predicates, `<coi ... mode="rank"/>` pointing at the written file, and `--generate-config`'s template and sorts. On the `counter` fixture (a mod-10 counter with enable):

```
$ harm-coi --top counter --files tests/input/coi/counter/rtl/counter.sv \
    --vcd-scope tb::dut --vcd-recursion 0 --vcd tests/input/coi/counter/trace.vcd \
    --predicates --emit-config $OUT/counter.xml -o $OUT/counter_coi.json
$ grep -E 'prop|coi' $OUT/counter.xml
<prop exp="cnt == 4'd0" loc="a, c, dt" origin="rtl"/>
<prop exp="cnt == 4'd9" loc="a, c, dt" origin="rtl"/>
<prop exp="en" loc="a, c, dt" origin="rtl"/>
<prop exp="rst" loc="a, c, dt" origin="rtl"/>
<coi file="counter_coi.json" mode="rank"/>
$ harm --vcd tests/input/coi/counter/trace.vcd --clk clk --vcd-ss tb::dut --vcd-r 0 \
    --conf $OUT/counter.xml --max-ass 3 --sva --dont-print-ass --dump-to $OUT/counter.txt > /
    dev/null
$ cat $OUT/counter.txt
always (cnt == 4'b1001 ##1 !(cnt == 4'b1001) |-> cnt == 4'b0)
always (cnt == 4'b1001 && en ##1 1'b1 |-> cnt == 4'b0)
always (rst ##1 1'b1 |-> cnt == 4'b0)
```

For GoldMine-style mining, the mode is changed to `filter`, with `depth` as needed.

6.4 Hints from the RTL (H22, D-036)

Predicates give HARM an alphabet, but every target still gets the same template, and every proposition may sit on either side of an implication. In trivergence an LLM supplies the rest from the specification: signal roles, where a proposition may go, templates. `--hints FILE.json` writes the same kinds of hint from the RTL, in a file an LLM could also write (`doc/schemas/hints.v1.json`; each entry says `origin: rtl` or `llm`), so that the two sources can be compared and merged. The ground rule is that hints shape roles and timing only, never which signal may explain which: a bug that cuts or adds a connection cannot hide an assertion, as the cone filter can.

- **Roles.** Each visible signal is an `input` (a top input port), a `reset` (named by `--reset`, or an asynchronous control), a `const`, a `clock`, `unknown` (as in the cone file), `comb` (a source at depth 0) or `state` (every source at depth ≥ 1), with a flag `control` when an RTL condition reads it.
- **Placement.** A predicate may be a consequent only if it reads a `comb`, `state` or `unknown` signal. An assertion whose consequent reads only inputs, resets or constants fails any formal check unless it is a tautology, so such predicates are antecedents only (`loc="a, dt"`).
- **Templates.** One per class of target: `G({..#1&..}|-> P0(1))` for combinational and unknown targets, `G({..#1&..}|=> P0(2))` for registers, which show their inputs one cycle later.

`--emit-config` then writes the configuration from the hints file, all of it (`--hints-use all`, the default) or the placement only (`--hints-use placement`). Without `--hints` the output is byte-identical to that of H10. An independent oracle recomputes the hints from `coi.json` alone; it agrees on the 7 fixtures and on all 21 designs of the miner portfolio's corpus. On the portfolio's measurement, the templates raised the proved assertions on every design that mines in the `filter` mode (`fsm` 8 to 25, `arbiter` 10 to 18).

6.5 Relations from the RTL (H24, D-038)

On the miner portfolio's D1 corpus (12 small designs with 49 planted bugs; a bug counts as caught when a formal tool refutes, on the buggy design, an assertion mined on the correct one), HARM caught 14 of the 37 bugs used for tuning. Traced bug by bug, the misses came from the alphabet, not the mining: signals that are only assigned had no proposition, data relations had none, 1-bit flags had one polarity, and reset assertions (`rst |=> ...`) are vacuous under the `disable iff (rst)` that trivergence's checks use. With `--hints` and `--emit-config`, `harm-coi` therefore also writes *relations* (`origin="rtl-relation"`), each with its own domain and template:

- an unconditional assignment as an equation, with SystemVerilog's widths kept (`wrap == (en && (cnt == 4'd9))`), under `G({!rst}|-> P0(5))` for combinational targets and `G({!rst}|=> P0(4))` for registers, whose right-hand side is read one cycle earlier with `$past`;
- a register's value at the end of reset, under `G({$fell(rst)}|-> P0(3))`;
- a conditional assignment's equation and, for a register assigned under a condition, `$stable(x)`, in single-cycle decision trees (`1D,3W,3A`) that find the condition;
- both polarities of a 1-bit flag.

On the `counter` fixture:

```
$ harm-coi --top counter --files tests/input/coi/counter/rtl/counter.sv \
    --vcd-scope tb::dut --vcd-recursion 0 --vcd tests/input/coi/counter/trace.vcd \
    --predicates --reset rst --hints $OUT/hints.json \
```

```
    --emit-config $OUT/counter.xml -o $OUT/coi.json
$ grep -E 'rtl-relation|fell' $OUT/counter.xml
<prop exp="wrap == (en &amp;&amp; (cnt == 4'd9))" loc="c, 5" origin="rtl-relation"/>
<prop exp="$stable(cnt)" loc="c, 7" origin="rtl-relation"/>
<template dtLimits="5D,3W,15A,-0.1E,U" exp="G({$fell(rst)}|-&gt; P0(3))"/>
$ harm --vcd tests/input/coi/counter/trace.vcd --clk clk --vcd-ss tb::dut --vcd-r 0 \
    --conf $OUT/counter.xml --sva --dont-print-ass --dump-to $OUT/counter.txt > /dev/null
$ grep -E 'past|stable|fell' $OUT/counter.txt | head -3
always (!rst && en |=> cnt == ($past((cnt == 4'b1001 ? 4'b0 : cnt + 4'b1)) & 4'b1111))
always (!en && !rst |=> $stable(cnt))
always (cnt == 4'b0 && en |=> cnt == ($past((cnt == 4'b1001 ? 4'b0 : cnt + 4'b1)) & 4'b1111))
```

Relations restate the RTL. They catch a design that has changed since it was mined, which is what trivergence and the portfolio's bug hunts ask; they cannot catch a bug that the mined RTL already has. With relations, HARM caught 34 of the 37 tuning bugs and 11 of the 12 held out, 45 of 49 (H24); 46 after H25 and 47 after H26, losing none it caught before; on the portfolio's three larger designs the mining loop took 1.1 to 2.5 times longer. Relations became the default with `--hints` at H24's sign-off; `--no-relations` restores the earlier output. Two oracles check them: every unconditional relation must hold at every cycle of the golden trace after reset, by an evaluator written for the test and, since H25, by HARM's own evaluation. Numeric hints (value clusters) were tried and dropped: on D1 they caught nothing the relations missed.

6.6 Signed RTL (H25, D-039)

A trace carries no signedness: HARM reads a VCD vector as an unsigned `logic`. `harm-coi` therefore wrote the signed comparison `sum > 13'sd2047` as `sum > 13'd2047`, true for every negative `sum`, and dropped comparisons with negative constants and relations with a sign extension. HARM now reads SystemVerilog's `$signed(e)` and `$unsigned(e)` (IEEE 1800-2017, 20.5): it evaluates them with D-035's rules, encodes them for Z3 and prints them as written. `harm-coi` writes a comparison of signed operands on `$signed` with a signed constant, a sign extension as `$unsigned( $signed(x) + 13'sd0)`, a signed shift as `$unsigned($signed(a) >>> n)`, and the RTL's own casts. The `$unsigned` around an extension matters: alone, `$signed(x) + 13'sd0` sits in the relation's unsigned context and is zero-extended, by SystemVerilog and by HARM alike. HARM evaluated that first rendering false on 289 of 502 samples, which the trace oracle, computing with unbounded integers, did not see; this is why HARM's own evaluation became the relations' second oracle. 163 expressions with casts agree with Icarus Verilog, and Z3 is checked on every 4-valued assignment of 9 pairs with casts.

6.7 Constants outside the reset branch (H26, D-040)

H10 made a proposition of a constant only in a reset branch. A constant assigned elsewhere, such as a saturated value (`acc <= 12'sd2047`), now gives `acc == 12'd2047` as a consequent of the relations' trees (H25's form when it is negative). A constant signal, a 1-bit target (both polarities exist already) and a target with more than 8 distinct constants (a lookup table: `aes_cipher` has 5,121) are left out. The trees take three propositions since H26: the guard that a formal tool required for the negative saturation, `en && !clr && $signed(sum) < -13'sd2047`, has three. Mining time was unchanged within noise.

6.8 Elaboration (H23, H28)

`harm-coi` gives slang the default timescale `--timescale 1ns/1ps`, as trivergence and OpenFPV do, so a design that mixes files with and without a `` `timescale `` directive elaborates (H28); a file's

own directive still governs it. HARM's keyword list (H23) is shared with `harm-coi`, which writes a signal named after a HARM keyword as an escaped identifier (`\W `).

7 Determinism and portability

v4 was developed on macOS (arm64) and checked on Linux (x86_64, Ubuntu 22.04), with the rule that both pass the same tests and mine the same assertions. Getting there exposed problems in the build, in the test infrastructure, and in HARM itself. Findings on Linux are numbered F-L1 to F-L10, on macOS F-M1 and F-M2.

7.1 One toolchain (D-010, H0)

At the start of the plan, HARM aborted on the development Mac even for `--help`: Spot was linked against GCC 12's C++ runtime, ANTLR and HARM against GCC 13's, and Boost against Apple's `libc++`, because Boost's build tool ignores `CC`/`CXX`. Before the fix, ctest even reported the example tests as passing while HARM was aborting at load time. Now every script in `third_party/` honours `CC`/`CXX` (Boost through an explicit toolset), records the compiler in `.harm_toolchain`, and CMake warns when HARM is configured with another one. On macOS, Homebrew's GCC 13 cannot compile against the newest Command Line Tools SDK, so the scripts use Xcode's SDK unless `SDKROOT` is set.

7.2 Linux (H11b, H11c, H11f)

- **F-L1: HARM did not build on Linux.** Z3's C++ API overloads `bv_val` for `int` , `unsigned`, `int64_t` and `uint64_t`. On LP64 Linux `uint64_t` is `unsigned long`, so the `unsigned long long` that the Z3 encoding passed matched none exactly; on macOS `uint64_t` is `unsigned long long`. The encoder now takes `uint64_t` (H11b).
- **F-L2:** the implication test, which enumerates all short traces for hundreds of pairs, took longer than ctest's default timeout on the slower Linux machine; it has a timeout of one hour.
- **F-L3:** the test registration asked yosys for `help read_slang`, which exits with 0 even when the command does not exist; the probe now runs `read_slang` on a one-line module (H11c).
- **F-L4** and **F-L5**, found on real designs, are D-028 (Section 2) and the 511-bit limit of harvested predicates (Section 6.3).
- **F-L9: the log files under concurrent writers** (H11f). HARM appends warnings and errors to `warning.log` and `error.log` as one JSON array, by reading the file, removing the last line and appending. On an empty file the line count underflowed (`size_t`), and a concurrent writer could empty the file between another's checks. Parallel ctest runs crashed intermittently, in the same tests as the crashes recorded as an open finding in H6. Writers now hold a file lock and a mutex for the whole read-modify-write, and an empty file is handled. Eight processes writing 200 warnings each now leave one valid array of 1,600 records. Since H21, a record is written over the array's closing line instead of rewriting the file, so a message costs the same however long the log is (2,000 warnings on a log of 50,000 records: 602 s before, 0.07 s after). The messages themselves were still printed outside any lock. In HARM's runs, messages from several threads could interleave on the console. In the test, whose eight threads printed into one string stream, the stream was corrupted and the test crashed now and then on Linux. Since H30, a message is printed whole under a console mutex, apart from the files' lock.

7.3 The test tools in `third_party` (H11d, H11e, H11g)

The oracles use Verilator, Icarus Verilog and yosys. v4 adds install scripts for pinned versions (Verilator 5.052, Icarus 13.0, yosys 0.69 with `read_slang`), and CMake prefers them over the ones on `PATH`, treating a Verilator older than 5 or a yosys without `read_slang` as missing. Moving to the pinned Verilator found that the fixtures depended on the simulator's version (D-029):

- **F-L7:** the fixture testbenches drew their stimulus from `$urandom`, whose sequence for a given seed changed between Verilator 5.031 and 5.052, so the committed traces could not be reproduced. The testbenches now use a xorshift generator written in SystemVerilog (`tests/input/stim.svh`), and a test checks that Verilator and Icarus produce the same stimulus.
- **F-L6:** the replay oracle negated property consequents with `!`, which IEEE 1800 does not allow (`not` is required); Verilator 5.052 rejects it.
- **F-L10:** Verilator 5.052 implements the end-of-simulation failure of a pending `s_eventually`, so the replay oracle now mines with `--trace-end sva`. It also never fails `a |-> not ( s_eventually p)`, so those controls are checked by a monitor validated on hand-labelled traces.
- **F-L8:** the influence oracle assumed both traces had the same signals; Verilator 5.052 no longer dumps a loop variable.
- **F-M1:** Verilator did not build on macOS, because Homebrew's `flex` is keg-only and its `FlexLexer.h` is on no include path; the scripts now put the Homebrew formulas' headers on `CPATH` (H11g).

HARM itself did not change in H11d, H11e or H11g.

7.4 The same assertions on arm64 and x86_64 (H12, F-M2)

After all tests passed on both machines, the evaluation's fixture table still differed on two designs: on `structs`, macOS mined 1,800 assertions and Linux 1,839, deterministically on each, for any thread count, from byte-identical configurations, and also with Linux built with the Mac's compiler version. Bisecting by build target, with floating-point contraction turned off in one target at a time, located the cause in two functions of the decision-tree search, the scores

$$cov = 1 - \frac{ATCT}{CT}\Big(1 - \frac{ATCF}{CF}\Big), \qquad H = -p\log_2 p - q\log_2 q.$$

On arm64, GCC contracts `a - b*c` into one fused multiply-subtract by default, which skips the rounding of the product; x86_64 without FMA instructions rounds it. The decision tree sorts candidates by gain and, with a small effort setting, keeps only the best. Among equal or nearly equal scores, the last bit decides which antecedent is kept, and so the tree, and the assertions. Of 19,182 coverage scores computed on arm64, 2,644 differed in the last bit from the step-by-step rounded value.

v4 compiles everything HARM's CMake builds with `-ffp-contract=off`. Linux is unchanged (it never contracted), and macOS now mines Linux's counts: the fixture table is equal on all 46 runs on both machines. Two tests guard it: the scores are compared bit for bit with a reference that rounds every operation, and `structs` must mine 1,839 assertions. A deeper fix, comparing gains with a tolerance and breaking ties by a fixed key, would remove the dependence on rounding altogether, but it changes the mined assertions on every platform; it was not taken (Section 9).

7.5 Smaller changes

- **`--version` prints `HARM <git describe>`** (H11, D-027), so users and trivergence can tell which build they run.
- **Profiling is opt-in** (H13): since v2, every GCC build was linked with `--profile` without compiling for profiling, so every run, even `--version`, wrote a useless 2.6 MB `gmon.out`. `-DHARM_PROFILE=ON` now builds a real `gprof` profile.
- **`--check-dump-eval` files** (H16, D-030): v3 named each checked assertion's CSV after its text with some characters deleted, so `a != b` and `a <= b` overwrote each other's file, and a name over 255 bytes stopped HARM. v4 writes `<k>_<text>.csv` and an `index.json` that maps each file to its exact assertion, and every row has all five columns. A test recomputes every row from the trace and compares it with the file the index names.
- **The proposition table** (H15, D-031): `--dump-prop-table` writes, for every context, each proposition's value at every sampled cycle, with the trace's files and reset segments, so that another miner (the SAT miner of the miner portfolio) can work on HARM's propositions with HARM's semantics, without a VCD reader or an evaluator of its own. Its tests compare every cell with check mode and with a VCD reader written for the test.
- **Findings left open by H15 and H16** (H17): `^`, `&`, `|` and `~` accept a CSV `bool` operand as a 1-bit value, as in SystemVerilog, marked as such when HARM types the variables, so that no parse that worked before changes (D-032); a float numeric's excluded values are compared with the values, not with the cycle index; `--vcd-dir` and `--csv-dir` concatenate the files in path order (D-033), so a multi-trace run numbers its cycles the same way on every machine.
- **Operators as in C and SystemVerilog** (H18, D-034): an audit of 1,154 expressions against Icarus Verilog, cross-checked with IEEE 1800-2017 and C11, found that HARM's `!` took a whole numeric comparison (`!x == y` read `!(x == y)`), that `& ^ \|` bound tighter than comparisons, that a Boolean compared with a number was compared as a truth value, and that some printed propositions re-parsed differently (`x - (y - k)` printed `x - y - k`). The grammar now follows the standard precedence, comparisons are numbers with a 1-bit result, unary `-` exists, and the printer uses the same table; expressions that parsed before keep their trees. The audit is now a test.
- **Evaluation as in SystemVerilog** (H19, D-035): signedness follows SystemVerilog (an operation with an unsigned operand is unsigned, also for C types), widths are context-determined (a comparison computes its arithmetic operands at its own width), decimal literals are 32 bits, shifts and divisions by zero never stop HARM (they stopped or aborted it before), `>>` is logical and `>>>` arithmetic, and literals print as they re-parse. A second fixture of 2,337 expressions, compared with Icarus Verilog under HARM's x/z rule, covers it; it found two Icarus pitfalls (a `?:` with an x condition, and `$isunknown` on some 2-state sums). After an optimisation of the comparisons, mining takes within 3 % of the time it took before (`bl_master10k`, `sobel1k`). The missing operators (`%`, `**`, reductions) are future work (H20).
- **Signals named after HARM keywords** (H23, D-037): a variable named `W`, `X`, `next` or another keyword no longer stops HARM; a formula writes it as a SystemVerilog escaped identifier (`\textbackslash W `), SVA output prints the plain name.
- **`harm-coi`'s hints, relations, signed RTL, constants and default timescale** (H22, H24–H26, H28) are described in Section 6.
- **Checked downloads** (H27): every third-party tarball is fetched over HTTPS with its certificate checked and kept only if its SHA-256 matches (`third_party/fetch.sh`); antlr4's

tag must be its known commit; the install scripts stop at their first error.

- **The Docker image** (`docker/build.sh`) builds a given git reference with Z3, `harm-coi`, Verilator and Icarus, and runs the fast tests while it builds.

8 Validation

Tests written together with the code tend to share its assumptions. Every semantic component of v4 was therefore checked against an *independent oracle*: something that computes the expected answer without HARM's code, or that checks HARM's answer with HARM's own evaluator from the other side. Table 1 lists them; `doc/plan/VALIDATION.md` has every run.

Table 1: The independent oracle of each component.

Component	Oracle	Result
Proposition semantics (H1)	Icarus Verilog computes, for 1,000 random expressions on 32 random 4-valued rows, both the SystemVerilog value and HARM's documented rule	0 mismatches with the rule; the SystemVerilog gap measured (D-011)
x/z rule (H1b)	23 pinned propositions, their SystemVerilog values checked against IEEE 1800 and with Icarus	differ on 9 of 23, as documented
SVA printing (H1, H1d)	Verilator replays HARM's mined assertions on the same trace: each must never fail, and a negated control of each must fail at least once	0 failures; all controls fire
Spot brackets and `->` (H1d)	Spot's `ltlfilt`: the new texts are equivalent to HARM's formulas, the old ones are not	pass
End-of-trace semantics (H1c)	a model of IEEE 1800's finite-trace semantics, written from the standard; 297 assertions × 4,680 traces × 2 modes	0 disagreements
Z3 equivalence (H2)	brute-force enumeration with HARM's evaluator: for 6-bit variables, all 4^6 four-valued assignments, 960 pairs; for wide variables, 20,000 random rows, 640 pairs. Every "not equivalent" must come with a counterexample that HARM's evaluator confirms	0 unsound; every equivalent 6-bit pair proved
Implication (H3)	every Boolean trace of 3 atoms up to length 6, judged by HARM's evaluator: a claimed $A \Rightarrow B$ is refuted if B fails where A does not; 310 generated pairs and 200 pairs in mined shapes, in both end-of-trace modes	0 unsound (after the D-004 amendment)
Atom premises (H3b)	2,000 random traces per pair in which the variables take `x`/`z` bits; the facts re-checked by enumerating 4,096 values	0 unsound; 10 of 10 facts agree

Component	Oracle	Result
Cone fixtures (H4)	simulation: toggling a signal claimed *outside* a target's cone, with the stimulus otherwise fixed, never changes the target; depths observed with one-cycle pulses	0 violations in 120 pairs; 102 of 105 depths observed (3 structurally unexcitable)
`harm-coi` (H5)	the hand-written fixtures; the simulation check above on its output; yosys bit-level reachability as a cross-check	exact on 6 fixtures; 0 unsound on 7 designs
Rank metrics (H6)	values computed by hand from D-014 on cones validated by simulation	pass, 24 assertions
Filter mode (H7)	per-consequent oracle: the union of rank runs on configurations restricted, in Python, to each consequent's cone	equal on 5 fixtures
Depth filter (H8)	the D-020 rule recomputed in Python from the printed text; soundness of every kept assertion; the D-007 table counted by hand	pass, 6 designs × 2 modes
Out-of-cone report (H9)	hand-written reports, and the rule recomputed in Python	pass
Predicates (H10)	hand-labelled predicates per fixture; Z3 finds no duplicates; no predicate is constant on its trace	pass, 26 predicates
Vector indices (H11c)	Verilator prints 14 selects on four vectors every cycle; HARM evaluates the same selects	equal on all 41 × 14
Same results on two platforms (H12)	the fixture table (46 runs) on macOS and Linux, compared count by count	46 of 46 equal
Proposition table (H15)	check mode's evaluation, and a VCD reader and evaluator written for the test, cell by cell	equal on every cell
`--check-dump-eval` files (H16)	every row recomputed in Python from the CSV trace	equal, 12 assertions × 8 rows
Operators (H18)	Icarus Verilog on 1,212 expressions (every pair of 17 binary operators, unary prefixes, 7 operand types), cross-checked with IEEE 1800-2017 and C11	861 agree on every row; the other 351 deferred to H19 by written rules
Evaluation (H19)	Icarus Verilog under HARM's x/z rule on 2,337 expressions (signedness, shifts, divisions by zero, x/z in arithmetic, literals)	pass; two Icarus bugs found and avoided
Hints (H22)	an oracle that recomputes roles, placement and classes from `coi.json` alone	equal on 7 fixtures and 21 corpus designs
Relations (H24, H25)	every unconditional relation evaluated at every cycle of the golden trace by an evaluator written for the test and, since H25, by HARM's own evaluation	hold on 5 fixtures; H25's first sign extension caught (false on 289 of 502 samples)
Sign casts (H25)	Icarus Verilog on 163 expressions with `$signed`/`$unsigned`; Z3 against every 4-valued assignment of 9 pairs	163 of 163; no unsound answer

Table 2: Mutation tests of the oracles: bugs planted in the implementation, and the tests that caught them.

Milestone	Planted bug	Caught by
H3b	facts proved without `x`/`z`	the hand-labelled facts
H3b	premises given to Spot but not to the finite model	the hand-labelled pairs, the command-line test
H3b	$p \to q$ stored as $q \to p$	the trace oracle (6 unsound claims)
H5	no control dependencies	fixtures, simulation, yosys
H5	register delay 0 instead of 1	fixtures (all 6)
H7	decision-tree candidates not pruned	4 of 5 fixtures
H8	distance off by one	unit tests and 12 end-to-end checks
H8	the tree's index ignored	11 end-to-end checks
H9	unknown variables counted as out	the unknown-signal fixture
H10	literal width + 1	4 fixtures, unit tests
H15	values written one cycle late	the VCD test, against both oracles
H18	the old precedence of `!`	the operator fixture (23 cases)

Component	Oracle	Result
Implication pairs (H29)	the kept sets of the earlier filter on the miner portfolio's corpus; a hand-counted fixture	equal in 40 of 40 runs; 58 pairs, as counted

8.1 Testing the oracles

An oracle that cannot fail proves nothing. For the components where it was possible, bugs were planted in the implementation, one at a time, and each oracle was run against them (Table 2). The mutation tests found real gaps, which were closed before the milestones were approved:

- in H7, "*any* variable in the cone" instead of "*every* variable" passed every fixture, because all fixture propositions had one variable. Two multi-variable propositions were added to each configuration;
- in H8, two planted bugs were caught only by unit tests, because no fixture had a plain `|=>` template or a distance beyond `max_depth`, and one end-to-end check compared HARM with its own metric. The fixtures gained two templates, and the check a Python rule;
- in H5, each family of oracles missed some bug the others caught: simulation and yosys cannot see depths or self-dependencies, and the fixtures cannot see constructs they do not contain. Only the constructs test caught all six.

8.2 Tests that were wrong

Writing tests first means some expectations turn out wrong. They were changed only with a recorded reason, and none to make an implementation pass. Examples: H5's blocking-temporary expectation (Section 6); an H1 expectation that encoded the SVA precedence bug (`s_eventually b_1 until b_2` for `F b_1 W b_2`); an H1c hand label (`!(b W c)` is a strong until, not safety); an H6 expected order with two equal scores in the wrong order (`X` sorts before `r` in ASCII, which is the D-001 tie-break).

8.3 The test suite

v3 had 35 tests; v4 has 277, run by `ctest -j8` in about 32 minutes on the Mac and 35 on the Linux machine. They include the frozen regression baselines of every example (H0), the determinism checks, the unit tests, the oracles above, and the documentation checks of this report (H14). Tests that need a tool (Verilator, Icarus, yosys, `harm-coi`'s Python) are registered only when it is found.

8.4 Not validated

- The planned cross-check of the reduction against USM-T's own evaluator was not run: USM-T builds only on Linux, and it would cover only the Boolean and integer subset that the hand labels already cover.
- No commercial simulator was available to check `s_eventually` and `not nexttime` at the end of a simulation; the oracle is the standard's semantics.

9 Limitations and future work

Known limitations.

- **Unknown values.** On cycles where a signal has `x` or `z` bits, HARM's verdicts can differ from a simulator's (Section 3.2). An option with SystemVerilog's semantics was withdrawn (D-024): it would break the complementarity of a proposition and its negation that the decision trees and the reductions rely on.
- **Reduction cost.** `--reduce implies` and `--atom-premises` grow with the square of the number of assertions. Since H29 only pairs that share a consequent atom are checked (on AssertLLM2's `i2c_slave`, 6,481 assertions: 1.2 M pairs instead of 5.7 M, and 125 s in all instead of 642 s on Linux), and above `--implies-max-pairs` (2,000,000 by default) the implication step is skipped for the context and the skip recorded, so a very large output keeps the assertions that `equiv` keeps. The pairs are checked one at a time: Spot's BDD dictionary is global and not thread-safe.
- **Ties in the decision tree** are decided by the last bit of a floating-point score. Platforms agree because they now round the same operations the same way (Section 7.4); a different `libm` could still change a choice under the entropy heuristic. Rounding-independent tie-breaking would change every platform's results, and was left for a later decision.
- **Cones are signal-level.** A write to one bit of a register makes the whole register depend on itself; the schema reserves a per-bit field that is not used. The out-of-cone report does not use depths.
- **Mining on real designs.** On the AssertLLM2 designs used in the evaluation, the decision-tree template that `--generate-config` writes, over every signal, timed out within 10 minutes on every design tried, so the evaluation there ran only the configurations built from the RTL's predicates (`harm-coi --emit-config`), which have far fewer propositions.
- **Older limits, unchanged.** Some formulas the grammar accepts cannot be evaluated (`F X X (a W c)`, for which Spot gives a non-deterministic automaton); the `camellia` example's configuration is not valid XML.

The evaluation and the release. The measurements of what each configuration does on the fixtures, the examples and ten AssertLLM2 designs (D-026) are in `eval/results/` and are

not part of this report. The release itself, merging `dev` into `main` with the tag `v4` (D-027), is deferred until the maintainers decide; until then, all fixes stay on `dev` (D-012).

Future work.

- the SystemVerilog operators HARM does not read yet: `%`, `**`, the reductions (`&v`, `|v`, ...) and `~^` (H20, planned in `doc/plan/H20_PLAN.md`), and the functions `$onehot`, `$countones` and `$isunknown`;
- checking implication pairs in parallel (in processes, each with its own Spot dictionary), so that large outputs need not reach the cap;
- depths in the out-of-cone report ("in the cone, but not at the specification's delay");
- a comparison with GoldMine on the designs it supports;
- diversity-aware selection under `--max-ass`, using USM-T's syntactic similarity;
- a shared core library for HARM and USM-T, so that the two do not drift further apart.

A Reference

A.1 New command-line options

No v3 option was removed or renamed. The new ones, all opt-in:

Option	Effect (section)
`--reduce syntactic|equiv|implies`	redundancy reduction; `syntactic` is v3's filter (Section 4)
`--keep stronger|weaker|ranked`	with `implies`: which side of an implication is kept (Section 4.2)
`--atom-premises`	with `implies`: facts between comparisons, proved with Z3 (Section 4.3)
`--atom-premises-max <N>`	at most N Z3 queries for those facts (default 2,000)
`--implies-max-pairs <N>`	with `implies`: skip the implication step for a context with more than N pairs (default 2,000,000; 0: no cap), recorded in `--dump-assertion-info` (Section 4.2)
`--dump-implications <file>`	the dropped assertions and what implies them, as JSON
`--dump-assertion-info <file>`	every kept assertion: metrics, leaves, cycle offsets, variables, origins, as JSON
`--dump-coi-report <file>`	the out-of-cone report (Section 5)
`--dump-prop-table <file>`	every proposition's value at every cycle, as JSON (`prop-table` v1, D-031)
`--trace-end harm|sva`	the end-of-trace semantics (Section 3.3)
`--skip-invalid-props`	skip a proposition that does not parse, with a warning
`--version`	print `HARM <git describe>` and exit

A.2 The options of `harm-coi`

Option	Effect (section)
`--top NAME`	the top module, the design under the trace's scope (required)
`--files F ...`	the SystemVerilog sources (required)
`--define NAME[=VALUE]`, `--include DIR`	preprocessor macros and include directories, repeatable
`--vcd-scope S`	the scope of the top instance in the trace, as HARM's `--vcd-ss` (required)
`--vcd-recursion N`	as HARM's `--vcd-r` (required)
`--vcd FILE`	the trace: its signal names are the visible ones (Section 6)
`--clock NAME`	the sampling clock, as HARM names it (default: inferred)
`--max-depth N`	the depth of the cones' closure (default 3)
`--edges FILE`	also write the direct edges
`-o`, `--output FILE`	the cone file, `coi.json` (required)
`--predicates`	harvest the RTL's conditions as propositions (Section 6.3)
`--emit-config FILE.xml`	write a ready-to-run HARM configuration
`--hints FILE.json`	write roles, placement and templates, and use them in `--emit-config` (Section 6.4)
`--hints-use all|placement`	with `--hints`: the configuration takes all the hints (default) or the placement only
`--reset NAME`	a reset input, repeatable (Section 6.4)
`--relations`, `--no-relations`	relations from the RTL; on by default with `--hints` and `--emit-config` (Section 6.5)
`-v`, `--verbose`	print why each signal is unknown and why each predicate was dropped

A.3 New configuration elements and metric variables

Element or attribute	Meaning
`<coi file="..." mode="rank"/>`	a cone file for the context; `mode` is `rank` (default) or `filter`
`<coi ... mode="filter" depth="exact"/>`	`depth` is `any` (default), `bounded` or `exact`; only with `filter`
`<prop ... origin="spec"/>`, `<numeric ... origin="..."/>`	free-text provenance, reported in the JSON dumps
`coiFrac`, `coiDepthFit`, `coiUnknown`	metric variables, available with a `<coi>` (D-014)
`G(P0)`	invariant templates, without an implication

The file format of `<coi>` is `doc/schemas/coi.v1.json`; the formats of the JSON dumps carry a `"version": "1"` field.

A.4 Decisions

Every decision of the plan, in `doc/plan/DECISIONS.md` unless noted.

Table 6: The decisions.

ID	Decision	Milestone
D-001	deterministic output: ranking ties broken by text, ordered collection	H0
D-002	SVA printing: `1'b1`, `|=>`, `##n`, `s_eventually`, `.`	H1
D-003	Z3 equivalence encodes HARM's 4-valued evaluator exactly	H2
D-004	implication only between safety assertions, in both infinite-word and HARM's finite-trace semantics (amended)	H3
D-005	COI depth = register crossings, sampled before the rising edge	H4
D-006	`harm-coi`'s front end is pyslang	H5
D-007	decision-tree index → cycle distance to the consequent	H8
D-008	by default the stronger assertion of an implication is kept	H3
D-009	Z3 is optional (`HARM_WITH_Z3`, default on) (`doc/plan/PLAN.md`)	H2
D-010	one C++ toolchain for HARM and every library	H0
D-011	x/z semantics of propositions: HARM's rule, documented	H1, H1b
D-012	fixes stay on `dev` until the release	—
D-013	COI sources and targets: visible signals, clock never a source	H4
D-014	the COI rank metrics	H6
D-015	mining keeps HARM's end-of-trace semantics; reduction requires both semantics	H3, H1c
D-016	`--trace-end sva`, the neutral view, as an option	H1c
D-017	"in the cone": every variable of the proposition	H7
D-018	COI metrics in filter mode; filter is not rank minus out-of-cone	H7
D-019	`harm-coi` conventions: too large, never too small	H5
D-020	depth modes of the filter	H8
D-021	SVA printing of `->`; brackets in Spot LTL	H1d
D-022	the out-of-cone report	H9
D-023	RTL predicate harvesting	H10
D-024	an SVA-semantics option for x/z: withdrawn (`doc/plan/H1b_PLAN.md`)	H1b
D-025	atom-implication premises	H3b
D-026	the evaluation's configurations C0–C7 (`doc/plan/H11_PLAN.md`)	H11
D-027	the release: tag `v4`, `--version`, Docker (`doc/plan/H11_PLAN.md`)	H11
D-028	vector indexing follows SystemVerilog	H11c
D-029	fixtures and oracles independent of the Verilator version	H11e
D-030	`--check-dump-eval` file names and `index.json`	H16
D-031	the proposition table, `prop-table` v1	H15
D-032	bitwise operators on `bool` operands	H17
D-033	trace directories in path order	H17
D-034	operators bind and convert as in C and SystemVerilog	H18
D-035	evaluation and conversions as in SystemVerilog	H19
D-036	hints from the RTL, `hints` v1	H22
D-037	signals named after HARM keywords: escaped references	H23
D-038	relations from the RTL (`harm-coi --relations`)	H24

ID	Decision	Milestone
D-039	signed RTL conditions and relations, `\$signed`/`\$unsigned`	H25
D-040	constants assigned outside a reset branch	H26

A.5 Milestones and findings

Table 7: The milestones from v3 to v4, with the findings fixed in each.

ID	Milestone	Findings fixed
H0	regression baselines; deterministic output	—
H1	proposition and SVA language	F9, F10 (H1's own)
H1b	x/z semantics, documented	—
H1c	end-of-trace strength, `--trace-end`	SVA precedence
H1d	SVA `->` and Spot brackets	F7, F8 (found in H8)
H2	Z3 back end, `--reduce equiv`	—
H3	`--reduce implies`	reduction time on deep assertions
H3b	atom-implication premises	—
H4	`coi.json` contract and RTL fixtures	—
H5	`harm-coi`	—
H6	COI rank mode, `origin`	—
H7	COI filter mode	template with too few propositions hung
H8	depth-aware filter	`->` offsets (F6)
H9	out-of-cone report	—
H10	RTL predicates, `--emit-config`	—
H11	evaluation, documentation, `--version`, Docker	—
H11b	HARM builds on Linux	F-L1, F-L2
H11c	yosys probe, vector ranges, wide predicates	F-L3, F-L4, F-L5
H11d	Verilator, Icarus and yosys in `third_party/`	—
H11e	fixtures and oracles independent of the Verilator version	F-L6, F-L7, F-L8, F-L10
H11f	log files under concurrent writers	F-L9
H11g	`install_verilator.sh` on macOS	F-M1
H12	the same assertions on macOS and Linux	F-M2
H13	profiling opt-in	—
H14	this documentation and report	—
H15	the proposition table, `--dump-prop-table`	—
H16	`--check-dump-eval` file names	overwrites, exit on long names (trivergence)
H17	bitwise operators on `bool`, float exclusions, sorted trace directories	float exclusions by cycle index
H18	operators and printing as in C and SystemVerilog	precedence of `!`, `& ^ |` and comparisons; printing that re-parsed differently
H19	evaluation and conversions as in SystemVerilog	shifts and divisions that stopped HARM; C's signedness rules for `logic`

ID	Milestone	Findings fixed
H20	(future work) missing operators	—
H21	leftovers from H19: log writes at a constant cost	logs rewritten whole on every message
H22	hints from the RTL (`harm-coi --hints`: roles, consequent placement, per-class templates)	—
H23	signals named after HARM keywords (`\textbackslash W `)	a keyword-named variable stopped HARM; `F`, `R`, `intersect` missing from the keyword list
H24	relations from the RTL (`harm-coi --relations`)	data outputs had no proposition, flags one polarity, reset assertions were vacuous under `disable iff`; fixed template propositions were cone-filtered
H25	signed RTL conditions and relations (`\$signed`, `\$unsigned`)	signed comparisons written unsigned; negative constants and sign extensions dropped
H26	constants assigned outside a reset branch as consequents	only the reset branch's constants were propositions
H27	pinned and checked third-party downloads (SHA-256, antlr4's tag commit, errexit)	downloads were version-pinned only, three without a certificate check; a failed download surfaced later as a CMake error
H28	`harm-coi`'s default timescale (`1ns/1ps`)	a design mixing files with and without `` `timescale `` did not elaborate
H29	the pairs of `--reduce implies` (a shared consequent atom; `--implies-max-pairs`)	any shared atom linked a pair, so the reset in every antecedent linked nearly all: `i2c_slave` spent 607 of 642 s there
H30	one message at a time on the console	messages were printed outside any lock: from several threads their characters could interleave, and a test pointing `std::cout` at one string stream crashed now and then on Linux
H31	the report brought up to date	`harm-coi`'s features since H10 were one-line bullets; limitations contradicted by H23 and H29; the PDF predated H21

References

[1] Giuseppe De Giacomo and Moshe Y. Vardi. Linear temporal logic and linear dynamic logic on finite traces. In *Proceedings of the 23rd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 854–860, 2013.

[2] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 4963 of *LNCS*, pages 337–340. Springer, 2008.

[3] Alexandre Duret-Lutz, Alexandre Lewkowicz, Amaury Fauchille, Thibaud Michaud, Etienne Renault, and Laurent Xu. Spot 2.0 — a framework for LTL and ω-automata manipulation. In *Automated Technology for Verification and Analysis (ATVA)*, volume 9938 of *LNCS*, pages 122–129. Springer, 2016.

[4] Cindy Eisner, Dana Fisman, John Havlicek, Yoad Lustig, Anthony McIsaac, and David Van Campenhout. Reasoning with temporal logic on truncated paths. In *Computer Aided Verification (CAV)*, volume 2725 of *LNCS*, pages 27–39. Springer, 2003.

[5] Samuele Germiniani and Graziano Pravadelli. Exploiting clustering and decision-tree algorithms to mine LTL assertions containing non-boolean expressions. In *2022 IFIP/IEEE 30th International Conference on Very Large Scale Integration (VLSI-SoC)*, 2022.

[6] Samuele Germiniani and Graziano Pravadelli. HARM: A hint-based assertion miner. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41(11):4277–4288, 2022.

[7] IEEE standard for SystemVerilog—unified hardware design, specification, and verification language. IEEE Std 1800-2017, 2018.

[8] Mike Popoloski. slang: SystemVerilog compiler and language services, and its Python bindings pyslang. https://github.com/MikePopoloski/slang. pyslang 12.0.0 is used.

[9] Wilson Snyder et al. Verilator. https://verilator.org. version 5.052 is used.

[10] Shobha Vasudevan, David Sheridan, Sanjay Patel, David Tcheng, Bill Tuohy, and Daniel Johnson. GoldMine: Automatic assertion generation using data mining and static analysis. In *Design, Automation & Test in Europe (DATE)*, pages 626–629, 2010.

[11] Stephen Williams. Icarus Verilog. https://github.com/steveicarus/iverilog. versions 12.0 and 13.0 are used.

[12] Claire Wolf et al. Yosys open synthesis suite. https://github.com/YosysHQ/yosys. version 0.69 with the read_slang front end is used.